**Experiment 2: Enforcing JSON/YAML Schemas**

**AIM:**

To generate valid structured data in JSON and YAML formats (e.g., from a small dataset containing student records), validate the generated output using a parser, and refine the prompt to produce syntactically correct and well-formatted structured data.

**DESCRIPTION:**

**JSON (JavaScript Object Notation)** and **YAML (YAML Ain't Markup Language)** are widely used data formats for storing and exchanging structured information. In prompt engineering, specifying an output schema helps Large Language Models (LLMs) generate responses in a consistent and machine-readable format, reducing formatting errors and improving interoperability with software applications.

In this experiment, prompts were designed to instruct the LLM to generate responses following predefined JSON and YAML schemas. The generated outputs were verified to ensure they contained the required fields, maintained the correct structure, and followed the specified format. The experiment demonstrated that schema-based prompting enables LLMs to produce reliable, well-structured, and application-ready outputs, making it useful for APIs, automation, and data processing tasks.

**PROGRAM:**

```
# Import libraries
from google import genai
import json

# Create Gemini client
client = genai.Client(
    api_key="YOUR_GEMINI_API_KEY"
)

# Prompt - JSON Output
response = client.models.generate_content(
    model="gemini-flash-latest",
    contents="""
Generate valid JSON for the following student records.

1. Name: Rahul, Age: 20, Department: CSE
2. Name: Priya, Age: 21, Department: AIML
3. Name: Arjun, Age: f2, Department: ECE

Return only JSON.
"""
)

print("===== JSON Output =====\n")
```

```
print(response.text)

# Validate JSON
try:
    json.loads(response.text)
    print("\nJSON is Valid")
except json.JSONDecodeError:
    print("\nJSON is Not Valid")
```

**OUTPUT:**

```
===== JSON Output =====

[
  {
    "name": "Rahul",
    "age": 20,
    "department": "CSE"
  },
  {
    "name": "Priya",
    "age": 21,
    "department": "AIML"
  },
  {
    "name": "Arjun",
    "age": "f2",
    "department": "ECE"
  }
]

JSON is Valid
```